

REPRESENTATION DES **ENTIERS** EN MACHINE

I) Notion de mot :

Les entiers vont être représentés par des emplacement mémoires formés d'octets (byte).

Un octet est un groupe de 8 bits (BIT = Binary digIT) dont la valeur est 0 ou 1.

Les systèmes informatiques manipulent des informations binaires et travaillent en général sur *une longueur fixe de bits* que l'on appelle *mot*.

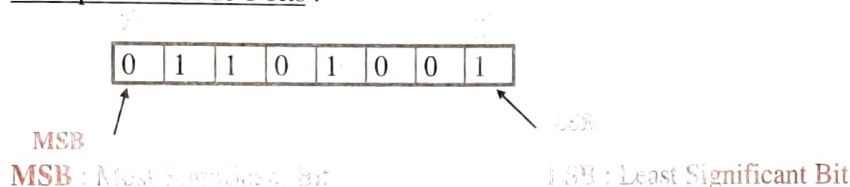
Le mot machine est l'entité de stockage de base, il est formé de 1, 2, 4, 8,... octets selon les machines c.à.d. 8, 16, 32 ou 64 bits.

II Les entiers non signés

1) Représentation

La représentation des entiers non signés en machine est dite à « *champ fixe* » car elle utilise un mot mémoire de taille fixe, 8, 16, 32 ou 64 bits donnée par le constructeur.

Exemple un mot de 8 bits :



A chaque bit, on associe une puissance positive de 2 appelée poids. Par convention ces puissances de 2 sont croissantes de la droite vers la gauche.

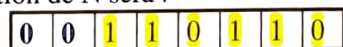
Ainsi le nombre entier non signé représenté ci-dessus est :

$$N = 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = (105)_{10}$$

Exemple1 : Quelle est la représentation machine de $N = (54)_{10}$ sur un octet (8 bits) ?

On convertit N en base 2. $N = (110110)_2$

La représentation de N sera :



On complète par des zéros à gauche

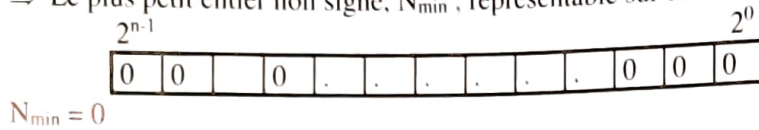
Exemple2 : Quelle est la représentation machine de $N = (363)_{10}$ sur un octet ?

On convertit N en base 2. $N = (101101011)_2$ **Nombre binaire à 9 bits !! Le nombre $(363)_{10}$ ne peut pas être représenté sur 1 octet.**

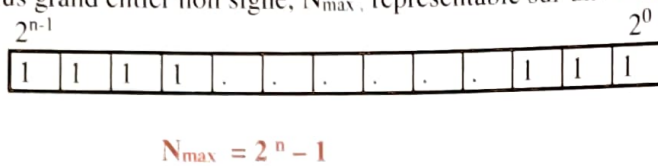
Attention à la saisie de nombres trop grands sous certains langages de programmation comme le C !!

Conclusion : L'intervalle des **entiers non signés** représentables sur un mot mémoire de n bits est limité :

⇒ Le plus petit entier non signé, $N_{\min}$, représentable sur un mot de n bits



⇒ Le plus grand entier non signé, $N_{\max}$, représentable sur un mot de n bits



Intervalle des valeurs codables en **NON signé** sur n bits : **$[0, 2^n - 1]$**

Ainsi on peut représenter sur :

1 octet les entiers non signés	$0 \leq N \leq 255$
2 octets les entiers non signés	$0 \leq N \leq 65\,535$
4 octets les entiers non signés	$0 \leq N \leq 4\,294\,967\,295$

2) Effet de dépassement de capacité lors d'opérations arithmétiques

• Addition sur 8 bits

	00101101	(45) ₁₀
+	00001110	(14) ₁₀
	00111011	

00111011 Pas de dépassement du format de représentation, pas de débordement de capacité. Le résultat est correct, soit (59)₁₀.

	01101101	(109) ₁₀
+	11001110	(206) ₁₀
	1 00111011	

1 00111011 Dépassement du format de représentation. La retenue engendrée sur le dernier rang binaire ne peut être représentée. **Il y a débordement de capacité.** Le résultat en machine est erroné, il est égale à (59)₁₀. !!!

La somme exacte dépasse la valeur de $N_{\max} = 2^n - 1$ et ne peut être représentée. En effet la somme exacte est égale à $(315)_{10} > (255)_{10}$.

⇒ En arithmétique **non signée**, une **retenue** engendrée sur le dernier rang lors d'une opération d'addition indique un débordement de capacité.

III) Les entiers signés (Entiers relatifs)

1) Convention de signe

Pour représenter le signe du nombre, le bit de poids le plus fort du mot binaire ou **MSB** est utilisé :



MSB = 0 pour indiquer que le nombre est **positif**
MSB = 1 pour indiquer que le nombre est **négatif**

2) Représentation en complément à 2

La plupart des constructeurs adoptent pour la représentation des entiers signés la représentation dite « *représentation en complément à 2* »

Avec cette représentation :

- Si le **MSB = 0**, l'entier est positif
- les entiers positifs sont représentés sous leur **forme binaire** comme pour un entier non signé

Exemple sur un octet, $(+27)_{10}$ est représenté par :



- Il n'existe qu'une seule représentation binaire pour la valeur zéro, qui est considérée comme une **valeur positive** :



- Si le **MSB = 1**, l'entier est négatif

Les entiers négatifs sont représentés « *en complément à 2* ». Cette représentation permet de donner une représentation convenable aux entiers négatifs de sorte à pouvoir toujours réaliser le calcul algébrique suivant :

$$\bullet \quad A - B = A + (-B)$$

↙ **Opposé de B**

Une soustraction pourra être ramenée à une addition

a) Notion de « complément à 2 »

Soit **N** un entier signé quelconque (positif ou négatif), représenté sur un mot de **n bits**. On appelle « complément à 2 » de **N**, le nombre noté $\overline{\overline{N}}$, tel que :

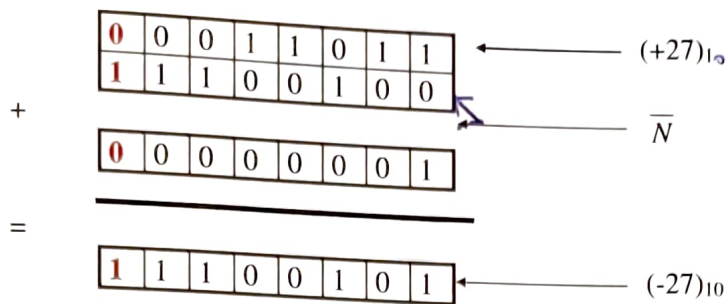
$$\overline{\overline{N}} = 2^n - N$$

Ce qui implique que

$$\overline{\overline{N}} + N = 2^n$$

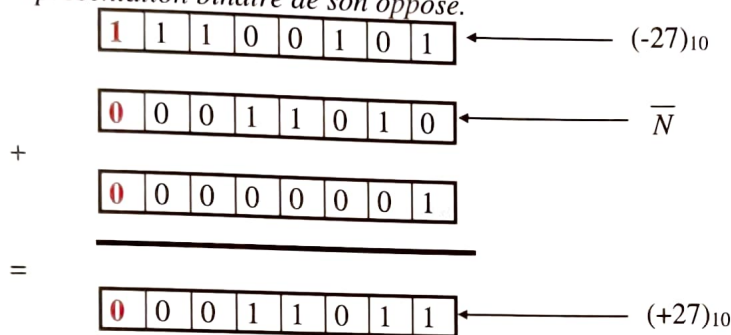
de n zéros précédés d'un 1, retenue engendrée, et qui est perdue car elle sort du format de représentation sur n bits :

Le Résultat est un **nombre binaire**



Remarques :

- On peut prendre « le complément à 2 » d'un entier négatif. On obtient alors la représentation binaire de son opposé.



- On peut obtenir la valeur décimale d'un entier négatif à partir de sa représentation binaire en affectant :

- un poids négatif au bit de signe
- des poids positifs aux autres bits

Ainsi la représentation binaire $(11000100)_2$ sur 8 bits d'un entier négatif a pour valeur décimale : $-1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = -128 + 64 + 4 = (-60)_{10}$

- A partir de la représentation binaire d'un entier signé sur 8 bits on peut déduire sa représentation binaire sur 16, 32 ou 64 bits par simple **extension** du bit de signe.

Exemple : $(+27)_{10}$ sur 8 bits 00011011 donne 0000000000011011 sur 16 bits
 $(-99)_{10}$ sur 8 bits 10011101 donne 1111111110011101 sur 16 bits

- On peut trouver directement l'opposé d'un entier signé, donné sous sa forme hexadécimale, sans avoir à passer par sa représentation binaire. Pour cela on revient à la définition première du complément à 2 en exprimant la relation $\overline{\overline{N}} = 2^n - N$ en hexadécimal.

Exemple : Soit l'entier signé $(FB37)_{16}$ représenté sur 16 bits en machine. C'est un nombre négatif. Quel est la représentation binaire de son opposé ?

$$\overline{\overline{N}} = 2^{16} - N = 16^4 - (FB37)_{16}$$

$$\begin{array}{r}
 \begin{array}{cccc}
 1 & 0 & 0 & 0 \\
 0 & & & \\
 F & B & 3 & 7
 \end{array}
 & \begin{array}{l}
 \nearrow 16 \\
 \text{codage de 2 puissance 16 en Hexadécimal}
 \end{array} \\
 - \\
 \hline
 0 \quad 0 \quad 4 \quad C \quad 9
 \end{array}$$

L'opposé de $(FB37)_{16}$ est $(04C9)_{16}$

Vérification en binaire :

$$\begin{array}{r}
 \begin{array}{cccc}
 1111 & 1011 & 0011 & 0111 \\
 0000 & 0100 & 1100 & 1000 \\
 0000 & 0000 & 0000 & 0001
 \end{array}
 & \begin{array}{l}
 \longleftarrow (FB37)_{16} \\
 \\
 \\
 \longleftarrow (04C9)_{16}
 \end{array} \\
 + \\
 \hline
 0000 \quad 0100 \quad 1100 \quad 1001
 \end{array}$$

c) Intervalle des entiers signés représentables en « complément à 2 » sur un mot mémoire de n bits

La représentation en complément à 2 ne permet de stocker qu'un nombre limité de valeurs selon la taille du mot dont on dispose.

⇒ Les entiers signés N représentables sur 1 octet sont donnée par l'intervalle :

$$-2^{8-1} \leq N \leq +2^{8-1} - 1$$

On en déduit que sur un mot de n bits on peut représenter tous les entiers signés données par l'intervalle :

$$-2^{n-1} \leq N \leq +2^{n-1} - 1$$

Ainsi sur :

- **16 bits** on représente tous les entiers signés : $-32\,768 \leq N \leq +32\,767$
- **32 bits** on représente tous les entiers signés : $-2\,147\,483\,648 \leq N \leq +2\,147\,483\,647$

3) Effet de dépassement de capacité lors d'opérations arithmétiques

• Addition de 2 entiers positifs codés sur 8 bits

$$\begin{array}{r} N1 \quad 00110011 \\ + \quad N2 \quad 01011100 \\ \hline \end{array}$$

$$\quad \quad \quad 10001111 \quad \leftarrow (-113)_{10} !!!$$

Le résultat en machine est négatif !! Il y a débordement de capacité

Cela est prévisible car $N1=+51$ et $N2=+92$ ce qui donne $N1+N2=+143$ qui est en dehors de l'intervalle de représentation des entiers signés sur 8 bits : $[-128,+127]$

Le résultat est donc erroné en machine. Ce **changement de signe est cependant indiqué par la machine.**

• Addition de 2 entiers négatifs codés sur 8 bits

$$\begin{array}{r} N1 \quad 11100000 \\ + \quad N2 \quad 10000001 \\ \hline \end{array}$$

$$\quad \quad \quad 101100001 \quad \leftarrow (+97 !!!)_{10}$$

retenue

Le résultat en machine est positif !! Il y a débordement de capacité

Cela est prévisible car $N1= -32$ et $N2= -127$ ce qui donne $N1+N2= -159$ qui est en dehors de l'intervalle de représentation des entiers signés sur 8 bits : $[-128,+127]$

• Addition de 2 entiers de signe opposé codés sur 8 bits

Aucun risque de débordement de capacité la somme donnera toujours un résultat compris dans l'intervalle $[-128,+127]$.

$$\begin{array}{r} N1 \quad 01011100 \quad \leftarrow (+92)_{10} \\ + \quad N2 \quad 10000001 \quad \leftarrow (-127)_{10} \\ \hline 11011101 \quad \leftarrow (-35)_{10} \end{array}$$

• Soustraction de 2 entiers signés

* Jamais de débordement de capacité quand il s'agit de soustraire deux entiers de même signe. En effet :

$$N1 - N2 = N1 + (-N2)$$

l'opération de soustraction est ramenée à une addition de 2 entiers de signe opposé.

* Risque de débordement quand il s'agit de soustraire des entiers de signe opposé. Vérifier le signe du résultat.

$$(+) - (-) = (+) + (+) = (+)$$

cette soustraction doit donner un résultat positif

$$(-) - (+) = (-) + (-) = (-)$$

cette soustraction doit donner un résultat négatif